

| 汇报时间     | 汇报人 |
|----------|-----|
| 2026.3.1 | 何宁秋 |

# 1. 阶段目标说明

## 近期任务

- 打基础阶段、资料收集

(1) 系统梳理 Vision-Language-Action (VLA) 的发展脉络，希望从研究范式的角度理解语言、视觉与机械臂控制是如何逐步融合的。

(2) 选择一个早期代表性方法 —— Language Policies —— 进行代码复现，从模块化结构出发理解语言如何影响控制策略。

## 阶段性目标

- 先建立对 VLA 技术路径的整体认知框架；
- 再从可解释、可控的经典方法入手理解语言条件控制的具体实现；
- 最后为后续向统一模型或生成式策略方向过渡打基础。

# 2. VLA 的发展脉络梳理

## 主要研究范式

我将 VLA 的发展大致归纳为四种模式：

- 模块化->端到端->Foundation 模型->生成式策略
- 趋势统一、规模大、数据及算力要求高

### P1: Language → Planning → Control (模块化阶段)

语言先被解析或编码，然后映射到规划或子策略，最终由控制模块执行。

特点是结构清晰、可解释性强。

代表工作包括 Language Policies、SayCan 等。

### P2: 视觉-语言端到端策略学习

图像与语言共同作为输入，直接输出动作。

多基于 imitation learning 或 offline RL。

代表工作如 PerAct、VIMA。

## P3: Foundation VLA

大规模数据训练统一模型，实现跨任务泛化。  
强调模型规模与数据规模。  
代表如 RT-1、RT-2、InternVLA。

## P4: 生成式策略 (Diffusion / World Model)

使用生成模型预测动作序列或隐变量轨迹。  
强调长时序建模能力。  
代表如 3D Diffusion Policy、WMPO。

## 当前技术瓶颈

- 机器人数据规模远小于 Web 级数据；
- 长时序任务建模仍存在困难；
- Sim2Real 泛化问题依然突出；
- 大模型算力和工程成本较高。

备注：数据规模、长时序任务建模以及真实环境泛化能力。

# 3. P1 模式典型案例复现 —— Language Policies

## 选择理由

主要基于以下考虑：

- 它是 P1 模式的代表性工作；
- 模块结构清晰，便于理解语言如何影响控制；
- 算力需求相对较低，适合作为研究起点；
- 可以作为后续方法对照的 baseline。

## Language Policies基础信息

### 项目与论文基础信息

| 项目        | 信息                                                                                                  |
|-----------|-----------------------------------------------------------------------------------------------------|
| 项目名称      | Language Policies                                                                                   |
| GitHub 地址 | <a href="https://github.com/ir-lab/LanguagePolicies">https://github.com/ir-lab/LanguagePolicies</a> |
| 论文标题      | Language-Conditioned Imitation Learning for Robot Manipulation Tasks                                |
| 发表会议      | NeurIPS 2020 (Spotlight)                                                                            |
| CCF 等级    | CCF A 类会议（人工智能方向）                                                                                   |
| arXiv 地址  | <a href="https://arxiv.org/abs/2010.12083">https://arxiv.org/abs/2010.12083</a>                     |

| 项目   | 信息                                    |
|------|---------------------------------------|
| 方法类别 | P1: Language → Policy (模块化)           |
| 学习方式 | Imitation Learning (Behavior Cloning) |

复现环境

| 类别           | 依赖 / 要求                                                                                         |
|--------------|-------------------------------------------------------------------------------------------------|
| 运行系统         | Ubuntu 22.04 (作者测试环境) ( <a href="#">GitHub</a> )                                                |
| Python       | Python 3.10 (作者测试环境) ( <a href="#">GitHub</a> )                                                 |
| 环境管理         | Conda env (environment.yml) ( <a href="#">GitHub</a> )                                          |
| 仿真/可视化 (机械臂) | CoppeliaSim (作者测试版本 4.1) ( <a href="#">GitHub</a> )                                             |
| 仿真接口         | PyRep (指定 commit) ( <a href="#">GitHub</a> )                                                    |
| 运动学库         | Orocos KDL + Python wrapper (指定 commit) ( <a href="#">GitHub</a> )                              |
| 评测通讯         | ROS2 (用于 live evaluation) ( <a href="#">GitHub</a> )-【实验没用到】                                    |
| 预训练模型/数据下载   | 作者提供的 Google Drive 文件 (包含 pre-trained model + 处理后数据等) ( <a href="#">GitHub</a> )                |
| 训练算力 (论文训练)  | GPU <b>非必须</b> ；作者报告在 <b>双 Intel Xeon E5-2699A v4 @ 2.40GHz</b> 节点训练 ( <a href="#">GitHub</a> ) |
| 训练时长 (论文训练)  | 约 35 小时 (与硬件相关) ( <a href="#">GitHub</a> )                                                      |
| 推理/展示算力      | README 未给出固定 GPU 型号/显存门槛；原则上 CPU 也可跑，GPU 可选 ( <a href="#">GitHub</a> )                          |

系统与软件依赖

| 类别        | 依赖                      |
|-----------|-------------------------|
| 操作系统      | Ubuntu (作者测试为 22.04)    |
| Python 版本 | Python 3.10             |
| 环境管理      | Conda (environment.yml) |
| 仿真软件      | CoppeliaSim             |
| 仿真接口      | PyRep                   |
| 运动学库      | Orocos KDL              |
| 通讯框架      | ROS2 (用于在线评测, 可选)       |

训练阶段 (论文报告)

| 项目       | 信息                  |
|----------|---------------------|
| GPU 是否必须 | 否                   |
| 作者使用硬件   | 双 Intel Xeon CPU 节点 |
| 训练时间     | 约 35 小时             |

## 方法结构及技术

### 核心结构

- 语言指令
- 语言编码 (LSTM)
  - 与当前状态拼接
  - Policy Network
  - 输出低层控制动作

数学形式可以理解为：

$$a = \pi_{\theta}(s, e_{lang})$$

本质是一个 language-conditioned policy network。

### 技术特点

- 使用语言 embedding 作为条件输入
- 基于 Behavior Cloning 训练
- 多任务共享参数
- 模块化结构清晰
- 不涉及大规模预训练

它代表的是早期“结构驱动”的 VLA 思路，而非“规模驱动”的 Foundation 模型。

## 创新及优缺点

### 创新点

在 2020 年背景下，其创新体现在：

- 将自然语言嵌入直接用于控制调节
- 实现语言条件 manipulation
- 在多任务场景下实现一定泛化能力

### 优缺点分析

#### 优点

- 结构清晰、可解释性强；
- 工程复杂度低；
- 易于扩展和修改；

- 适合作为可控 baseline。

缺点

- 泛化能力有限；
- 强依赖人工结构设计；
- 不具备 Foundation 模型的规模优势；
- 难以直接扩展到大规模真实机器人数据。

## 4. 下一阶段研究计划

(P3 / P4 路线推进)

总体目标

从模块化 P1 baseline 过渡到：

- P3: Foundation VLA 统一模型理解与复现
- P4: 生成式策略 (Diffusion / World Model) 探索

路径：

Baseline 理解 → Foundation 复现 → 生成式策略探索

### 阶段规划总表

| 任务类型                     | 任务内容                                           | 相关地址                                                                                                                                                                                                                           | 计划开始    | 计划完成    |
|--------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|---------|
| P3 - Foundation VLA 复现   | 阅读并复现 RT-1 (理解 Robotics Transformer 架构与数据组织方式) | GitHub: <a href="https://github.com/google-research/robotics_transformer">https://github.com/google-research/robotics_transformer</a> <br> 论文: <a href="https://arxiv.org/abs/2212.06817">https://arxiv.org/abs/2212.06817</a> | 2026.03 | 2026.04 |
| P3 - Foundation 数据理解     | 研究 Open-X Embodiment 数据格式与多机器人统一建模方式           | GitHub: <a href="https://github.com/google-deepmind/open_x_embodiment">https://github.com/google-deepmind/open_x_embodiment</a> <br> 论文: <a href="https://arxiv.org/abs/2310.08864">https://arxiv.org/abs/2310.08864</a>       | 2026.04 | 2026.05 |
| P3 - 统一模型结构对比            | 复现 InternVLA 或 CogACT, 分析与 RT-1 的结构差异          | InternVLA: <a href="https://github.com/InternRobotics/InternVLA-M1">https://github.com/InternRobotics/InternVLA-M1</a> <br> CogACT: <a href="https://github.com/microsoft/CogACT">https://github.com/microsoft/CogACT</a>      | 2026.05 | 2026.06 |
| P4 - Diffusion Policy 复现 | 复现 3D Diffusion Policy, 理解生成式动作建模流程            | GitHub: <a href="https://github.com/YanjieZe/3D-Diffusion-Policy">https://github.com/YanjieZe/3D-Diffusion-Policy</a> <br> 论文: <a href="https://arxiv.org/pdf/2403.03954">https://arxiv.org/pdf/2403.03954</a>                 | 2026.06 | 2026.07 |
| P4 - World Model 探索      | 阅读并尝试复现 WMPO, 理解世界模型优化机制                       | GitHub: <a href="https://github.com/WM-PO/WMPO">https://github.com/WM-PO/WMPO</a>                                                                                                                                              | 2026.07 | 2026.08 |

| 任务类型        | 任务内容                                             | 相关地址                     | 计划开始    | 计划完成    |
|-------------|--------------------------------------------------|--------------------------|---------|---------|
| P4 - 结构对比实验 | 对比模块化(P1)、Foundation(P3)、Diffusion(P4)在长时序任务上的表现 | 基于 CALVIN 或 RLBench 统一评测 | 2026.08 | 2026.09 |

## 细化任务说明

### 第一阶段（P3 方向：统一模型理解）

目标：

- 理解 Robotics Transformer 如何建模多任务
- 理解数据 token 化方式理解 Action token 表示方法

输出成果：

- 架构对比分析文档
- 模型流程图
- 与 P1 的结构对比总结

### 第二阶段（P4 方向：生成式策略）

目标：

- 理解 Diffusion Policy 如何生成连续动作序列
- 理解 latent action 表示
- 分析其在 long-horizon manipulation 中的优势

输出成果：

- Diffusion policy 训练流程图
- 与 BC policy 的对比实验
- 长时序性能分析报告

## 预期技术路线图

P1（模块化 baseline）

↓

P3（统一模型，数据驱动）

↓

P4（生成式策略，长时序建模）

整体路径是：

模块化解理解 → 结构增强 → Foundation 模型 → 生成式策略探索。